

HPC-QPU Hybrid

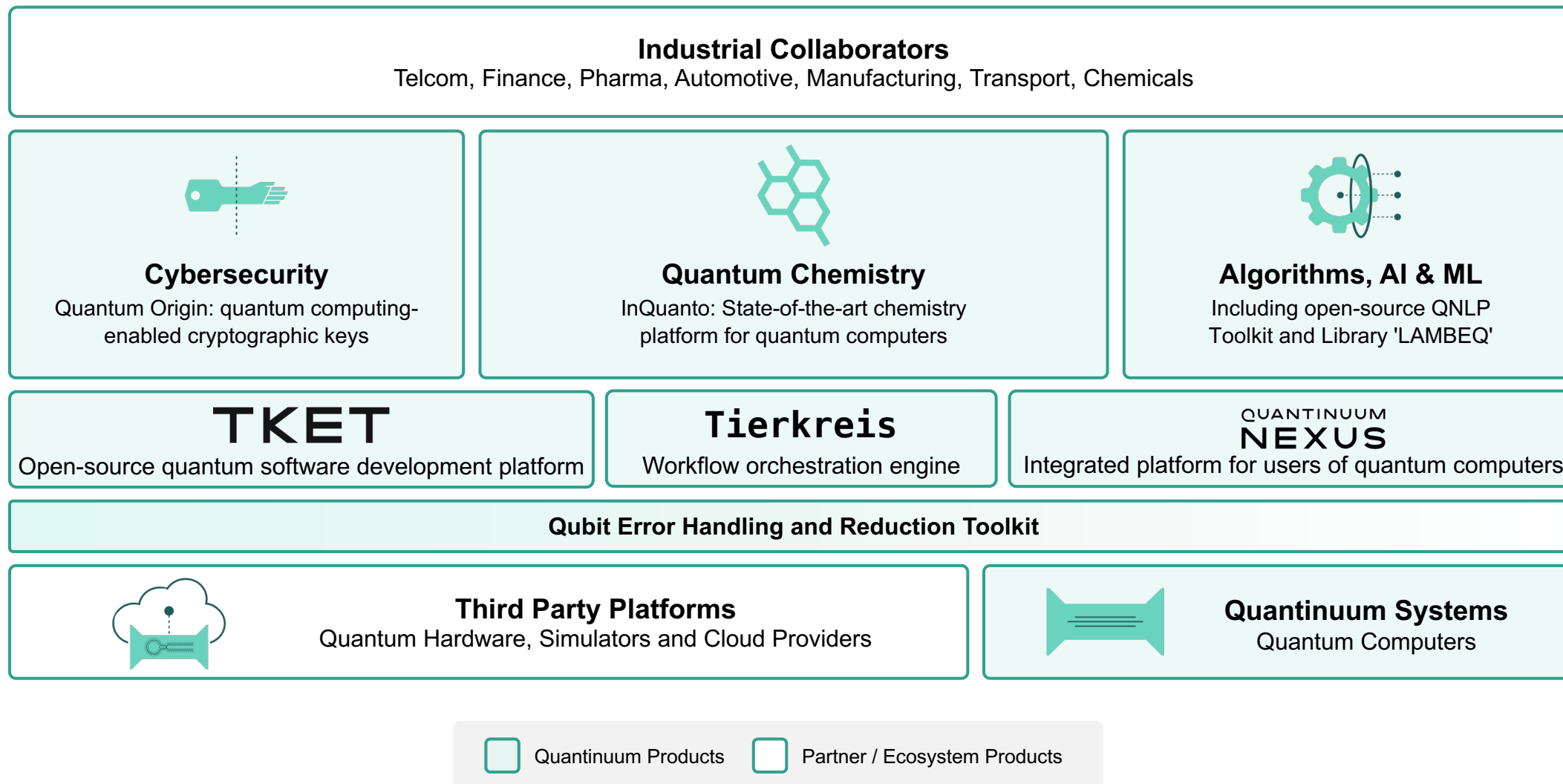
Alex Chernoguzov, Neal Erickson, John Children

04/13/2026

Outline

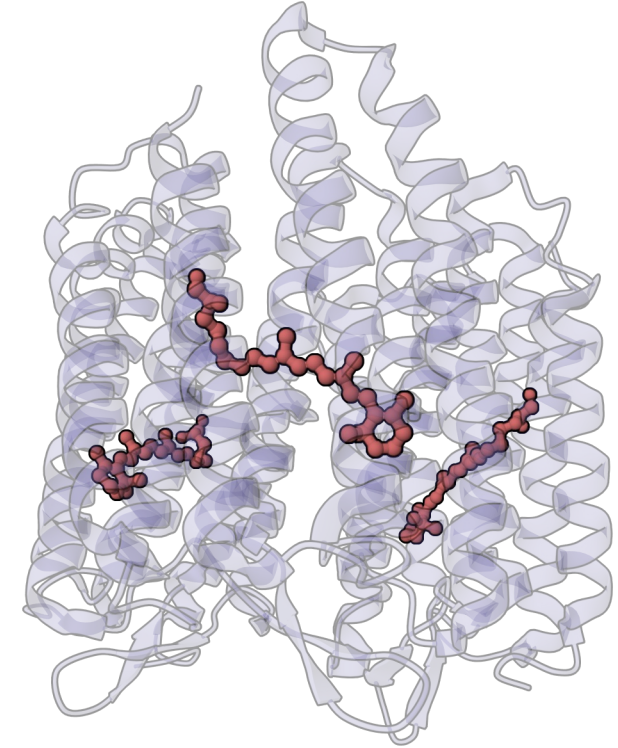
- Introduction
- Why hybrid – motivation and challenges
- Job submission and scheduler control
- Workflow management

Quantinuum Stack



Why HPC-QPU Hybrid?

- Intuition
 - Classical HPC systems are good at modeling nature at macro scales
 - Quantum computers are good at modeling nature at micro scales
 - Combination of the two enables better, more accurate, faster to compute models
- Example
 - Selective photo-isomerization of retinal embedded in bacteriorhodopsin
 - Environment determines reaction properties
 - More details in [arXiv:2601.15677](https://arxiv.org/abs/2601.15677)

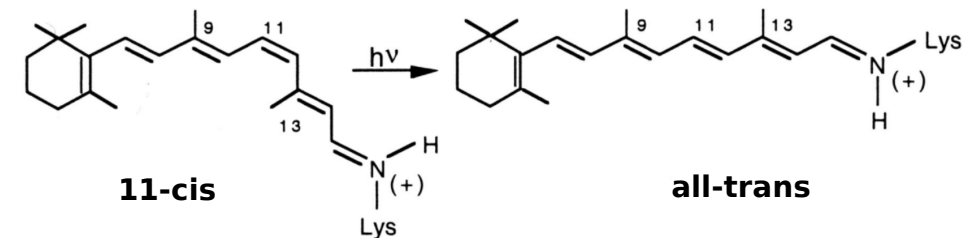


Environment (protein backbone)

Large size, weak electron correlation → HPC

Reaction center (embedded retinal)

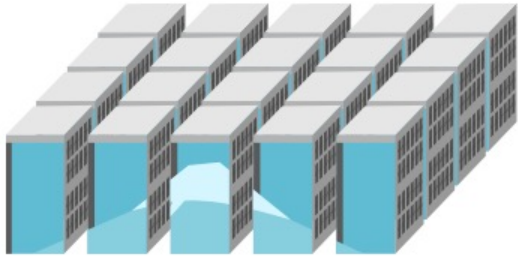
Small size, strong electronic correlation → QPU



Core Beliefs

- HPC and QPU will be deployed side by side to solve industrially relevant computational problems
- HPC and QPU operate on similar time scales to perform useful computation – seconds to hours
- Hybrid applications require complex interactions between HPC and QPU
- HPC and QPU are expensive and rare resources
- Hybrid systems are intrinsically multi-user
- QPU is self-contained

Hybrid Scheduling Goals



Scaling Dimensions:

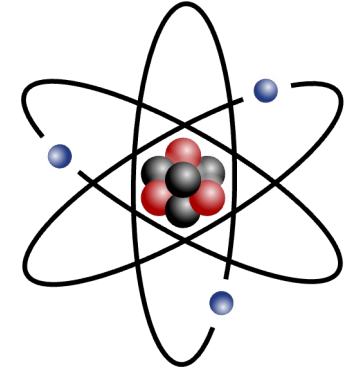
- CPU/GPU compute cores
- Memory
- Storage

Large number of nodes, cores

Multiple scheduling dimensions

Common Scheduling Goals

- Optimize resource utilization
 - Maximize job throughput
 - Minimize downtime
- Provide fair multi-user access



Scaling Dimensions:

- Qubit count
- Operation fidelity
- Execution speed

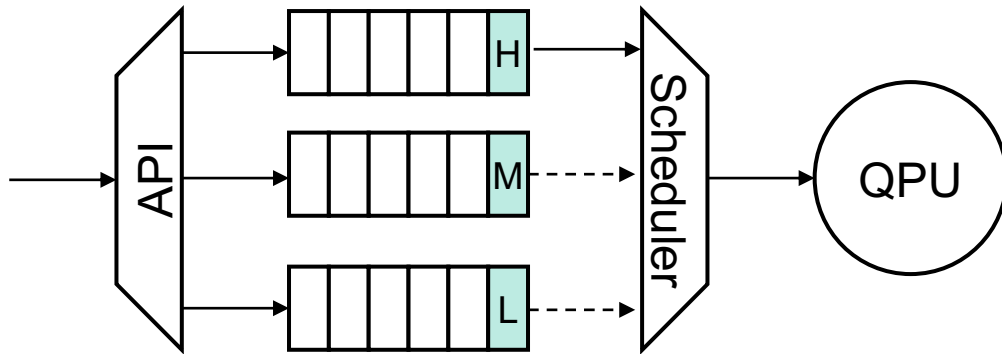
One/two QPU's per cluster

One scheduling dimension – time

Quantum Job Scheduling Options

Priority job queues

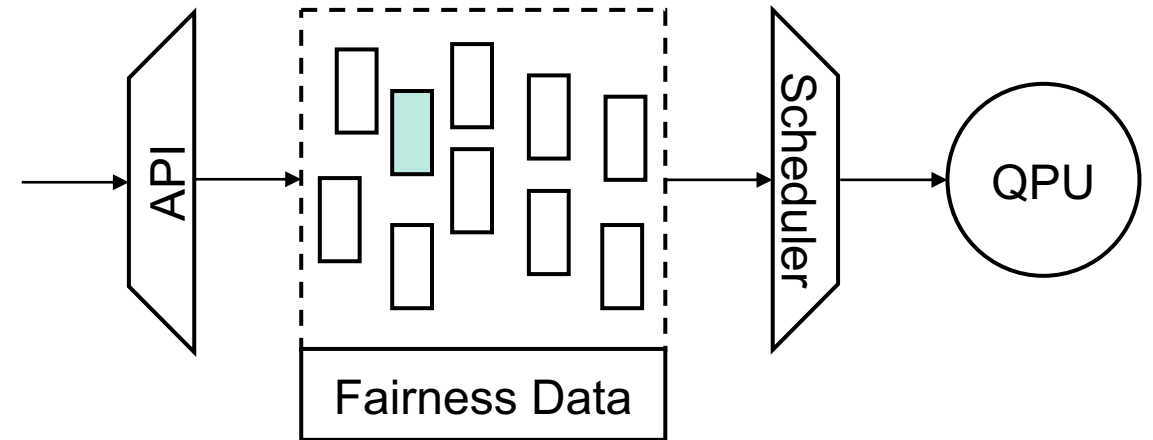
- + Simple to understand queue order
- + Standard OS scheduling algorithms
- No concept of fairness
- Limited flexibility
- Job-level granularity



static priority queuing

Adaptive fair queueing

- + Job selection just in time
- + Flexible set of job selection criteria
- + Job slice granularity
- No concept of queue order



dynamic priority queuing with time slicing

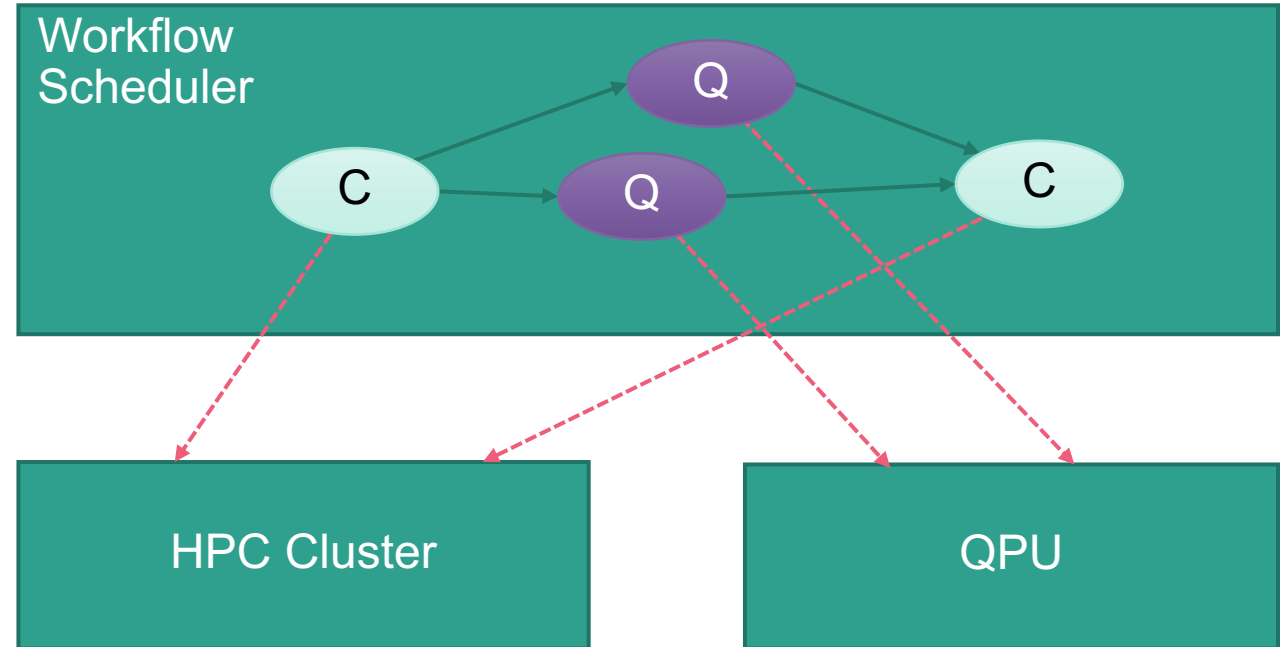
Hybrid Co-Scheduling Challenges

- HPC-centric viewpoint
 - I want QPU to be available immediately on-demand
 - QPU idles waiting to be called on by the HPC
 - HPC idles waiting for QPU to provide the answers
 - Big dog syndrome
- QPU-centric viewpoint
 - I need to process multiple jobs from multiple users concurrently
 - HPC cluster is often not the only QPU submission channel
 - Quantum scheduling is inherently different from HPC scheduling
 - Aspiring big dog syndrome
- A better way
 - Hybrid workflow system works with existing schedulers
 - HPC and QPU resources managed side by side
 - HPC and QPU resources allocated on demand



Coordinating Hybrid Jobs

- Workflow scheduler
 - Lives outside of the HPC cluster
 - Delegates work to HPC nodes
 - Delegates work to QPU nodes
 - HPC nodes perform classical computation tasks
 - QPU nodes perform dedicated quantum tasks
-
- Key performance drivers
 - Multiple classical and multiple quantum tasks can run concurrently
 - Data dependencies trigger classical-quantum handoff



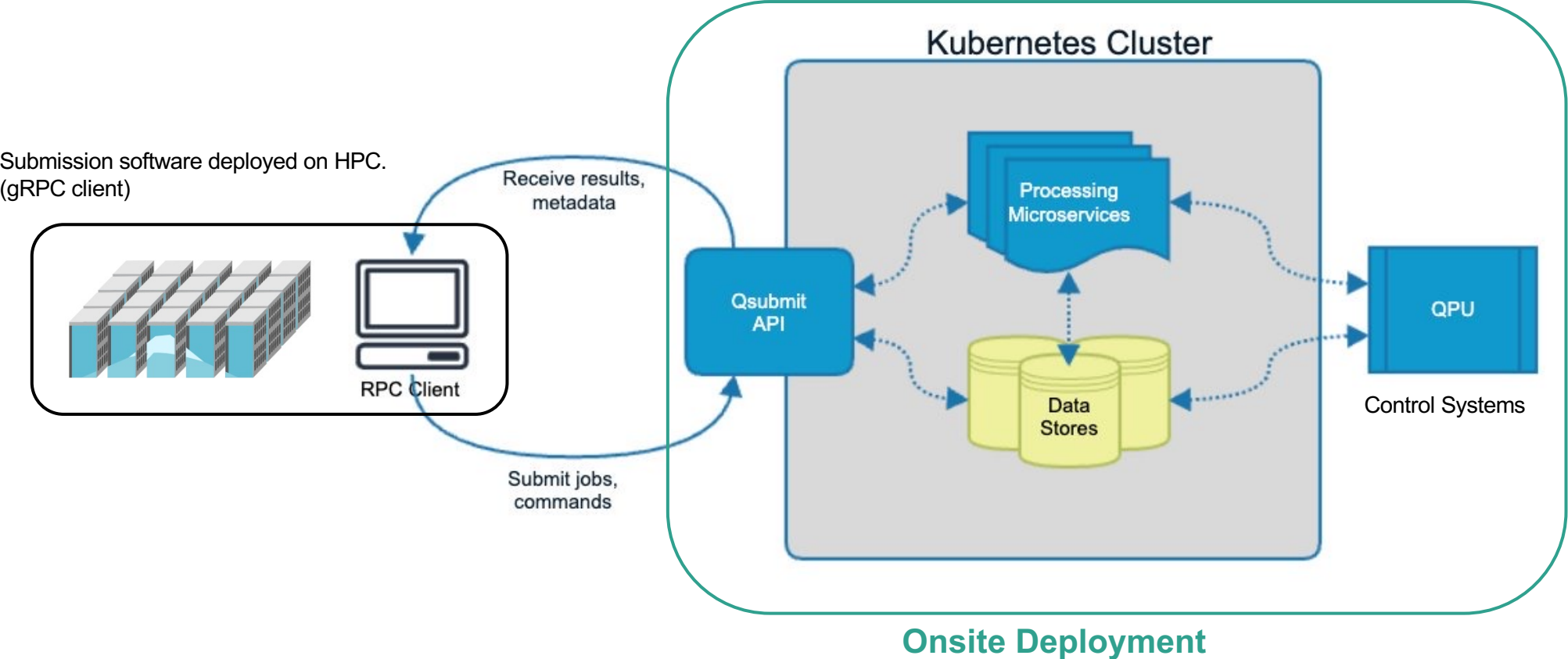
Job Submission

Managing QPU job submissions via RPC protocol

Need for a Direct Access API

- With a co-located HPC/QPU, it is undesirable to access the QPU via existing cloud APIs
- A direct access API:
 - Prevents latency from the network overhead & unnecessary processing
 - Provides resilience against network & cloud provider outages
 - Has API shape optimized for hybrid execution usage, rather than standard

Qsubmit Deployment



QSubmit API Details

- gRPC (Protobuf-based RPC) service written in Go
- Hosted in on-premises Kubernetes cluster
 - Nearby QPU control system hardware
- Uses mTLS authentication
- Provides streamlined, hybrid-focused API:
 - Job submission & execution methods
 - Streaming or synchronous results fetching
 - Ahead-of-time compilation, triggered execution workflow
- Deployed at RIKEN

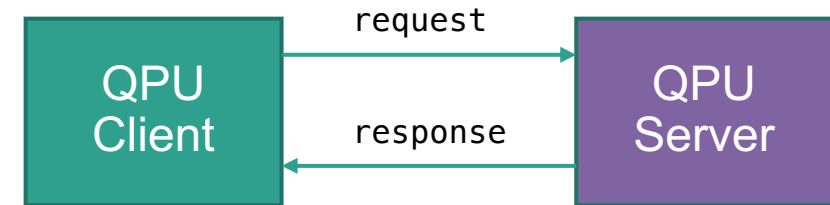
RPC, But What RPC?

- Many RPC variants, well established
- We propose **gRPC**:
 - Modern, well supported, good tooling
 - Multi-language support
 - Simple service definition
 - Asynchronous mode
 - Streaming support
 - Secure channel
- Protobuf serialization
 - Compact binary encoding
 - Required and optional fields
 - Versioning

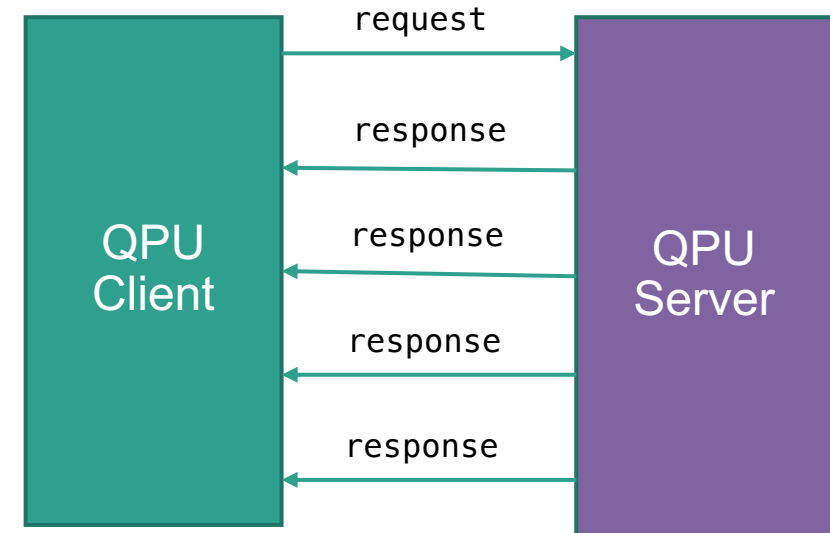
<https://grpc.io>

<https://protobuf.dev>

Traditional request/response

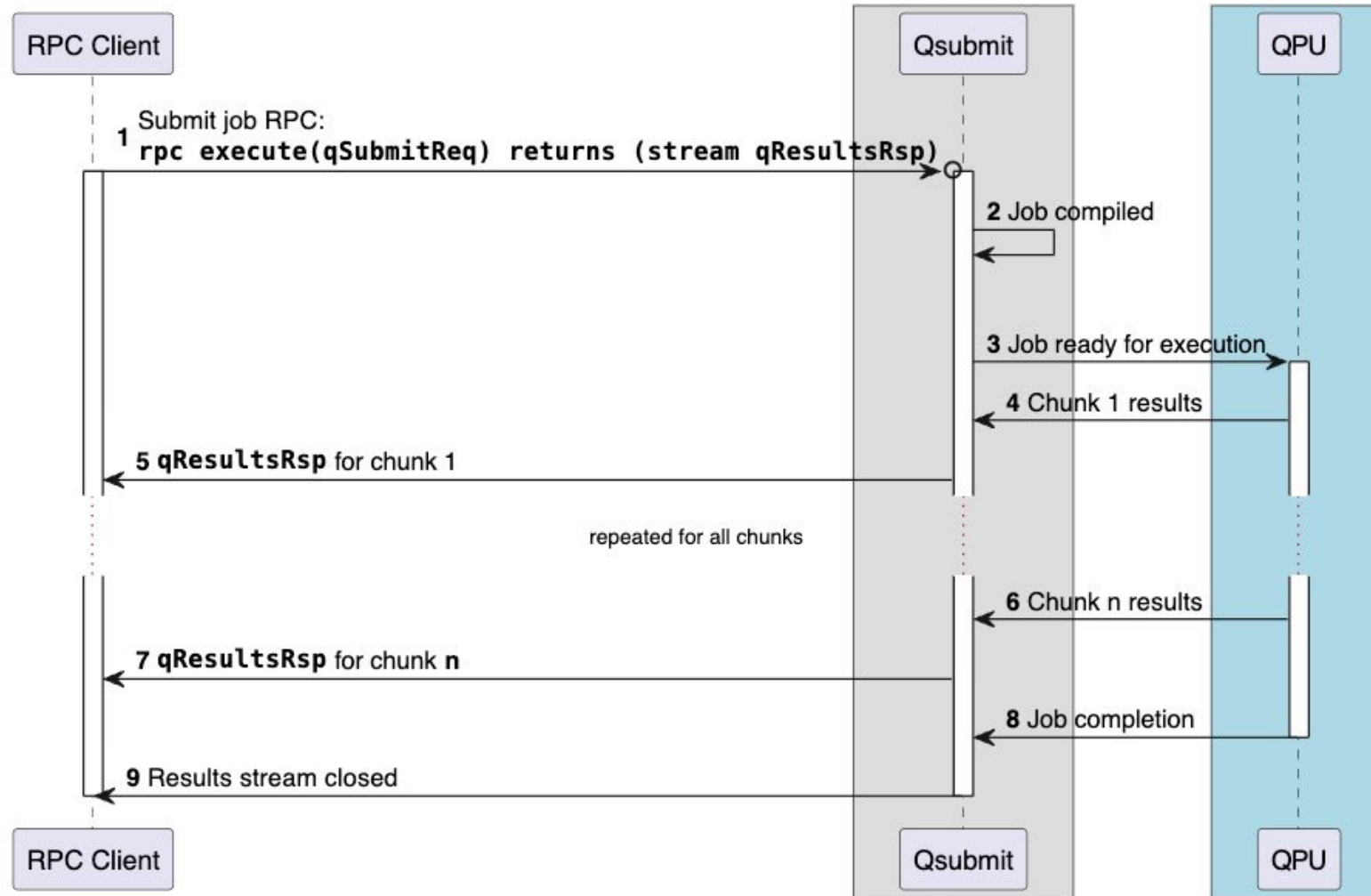


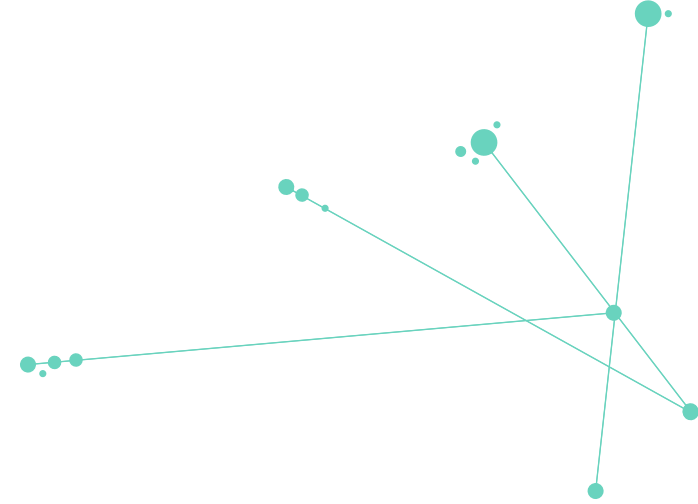
Server-side streaming



Example of Simple Submission Workflow

(many other RPC methods exist!)





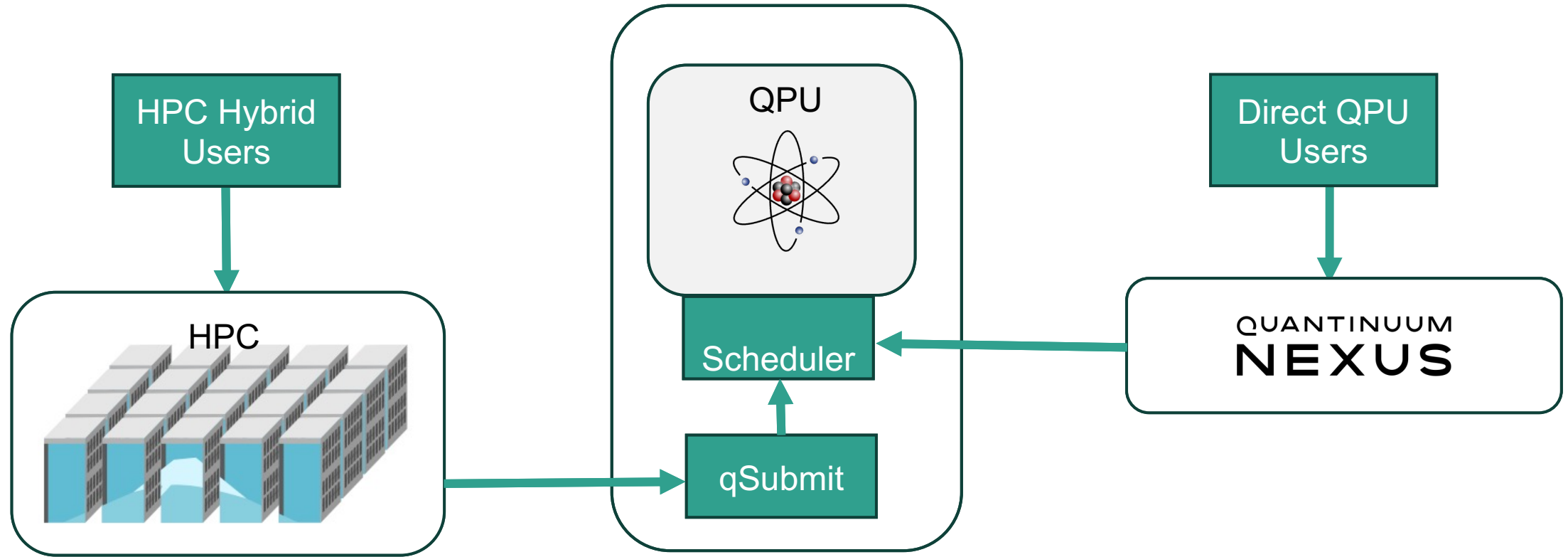
QPU Scheduling

Scheduling algorithm and scheduler control

Scheduling Needs

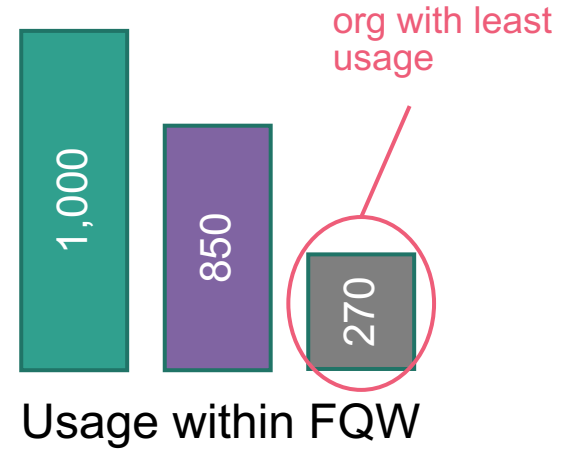
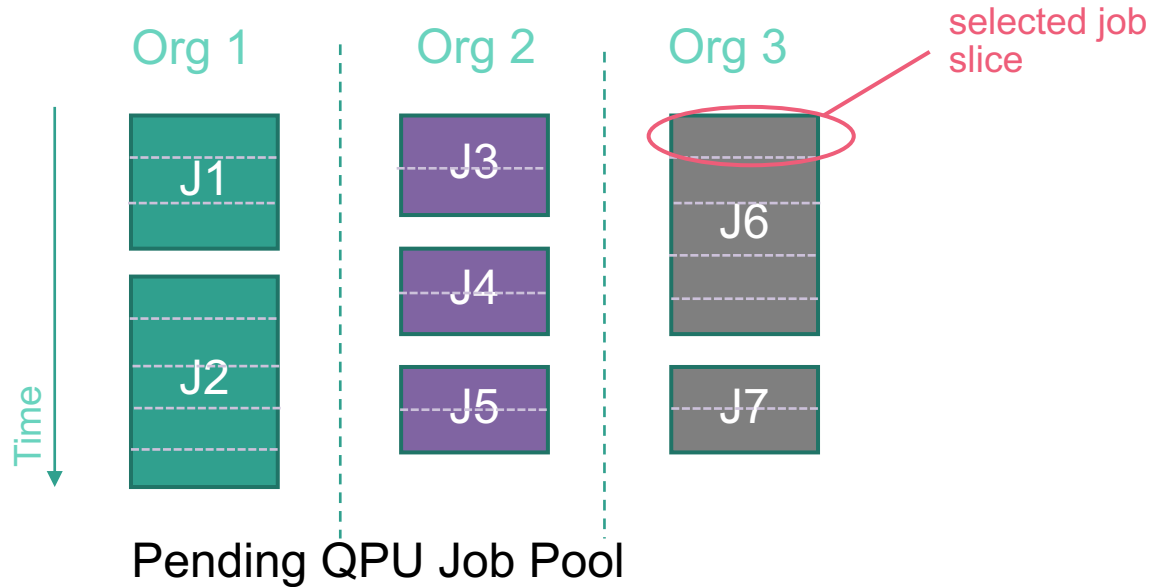
- Hybrid execution jobs need to take priority over "standard" jobs
 - Want to minimize time during hybrid execution, as both HPC and QPU are committed
- Leverage existing scheduling capabilities
 - Work with "Fair Queueing" system already in place
- Provide level of customization to operator's preference
- Minimize "wasted" time on both HPC and QPU

QPU in HPC Context and Standalone



- Fair queuing QPU scheduler is used for both hybrid and direct quantum jobs
 - HPC jobs run in the higher priority class than direct QPU jobs via org, group, user configuration

"Fair Queueing": QPU Scheduler Algorithm



1. Select **organization** with least usage within FQW
2. Select highest priority **group** within selected org
3. Select highest priority **user** within selected group
4. Execute one **slice** of selected **job**
5. Update FQ accumulation (usage)

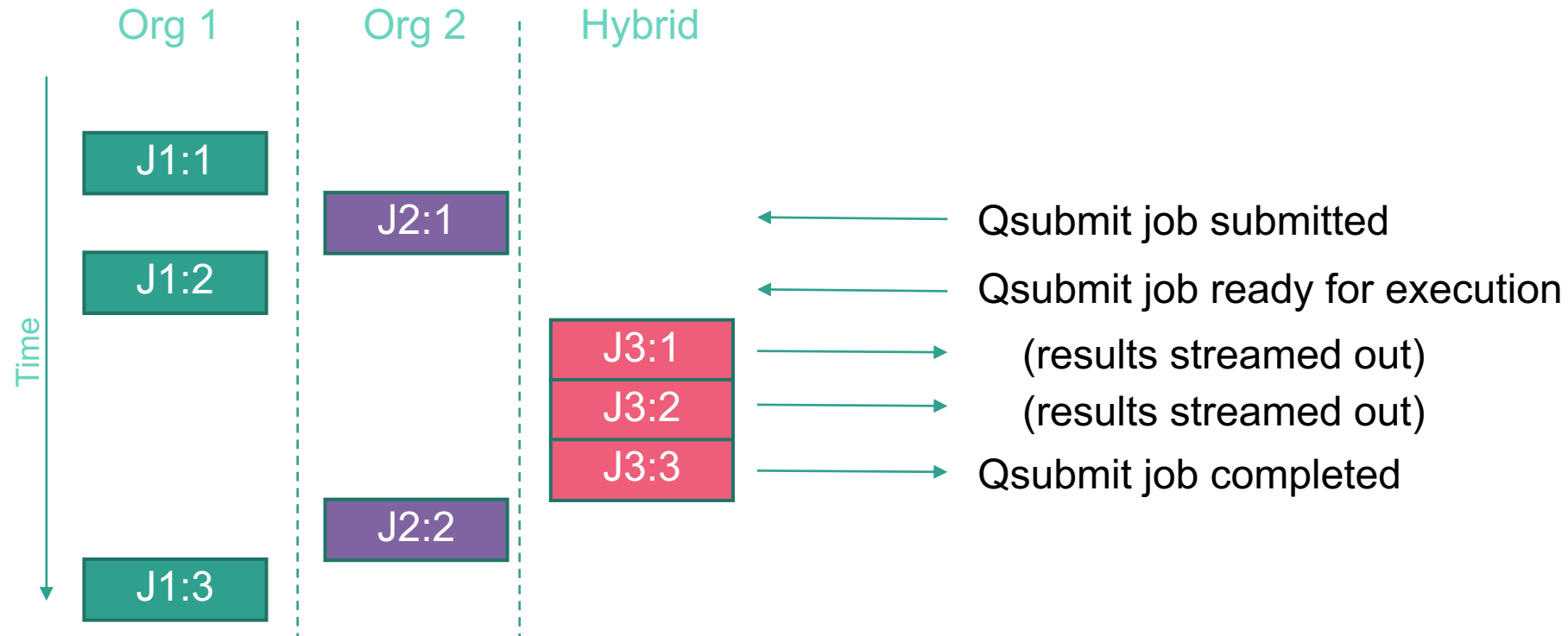
- Steps 1-5 repeated until the end of FQW
- Start new FQW – reset accumulators

FQM is used to skew fairness – prioritize orgs
Time slicing increases fairness

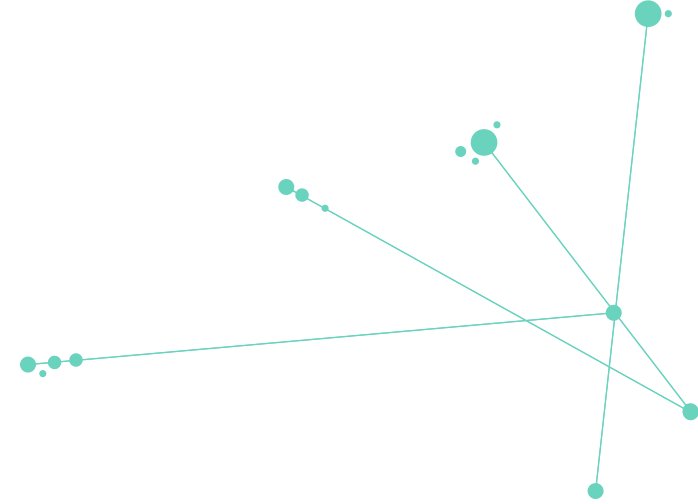
Qsubmit Scheduling

- Qsubmit service is configured to submit jobs with given priorities
 - Both an internal high and low option are configured
- Enables interruption of less urgent work in favor of hybrid execution jobs
 - Previous work is interrupted safely at a slice/chunk switch, not tossed

Qsubmit Submission Example



Pending QPU Job Pool



Workflow Orchestration

Tierkreis Workflow Management System

Why build a new Workflow Management System?

Many existing WMS tools do not meet the needs of hybrid Quantum-HPC:

- HPC and Quantum integration is usually an afterthought
- Not all WMS tools support loops and/or control flow that are needed for variational workflows
- Heavy python bias for classical compute that is undesirable in HPC for efficiency
- The programming interface for the Workflows is often unwieldy and error prone
- It can be difficult to see what your Workflow is doing and debug it

Tierkreis Design Principles

- Re-use existing HPC tooling as much as possible
- Minimize CPU and QPU idle time
- Provide a friendly programming interface
- Support control flow and loops in Workflows
- Provide tools to observe Workflows during execution

Tierkreis Program Model

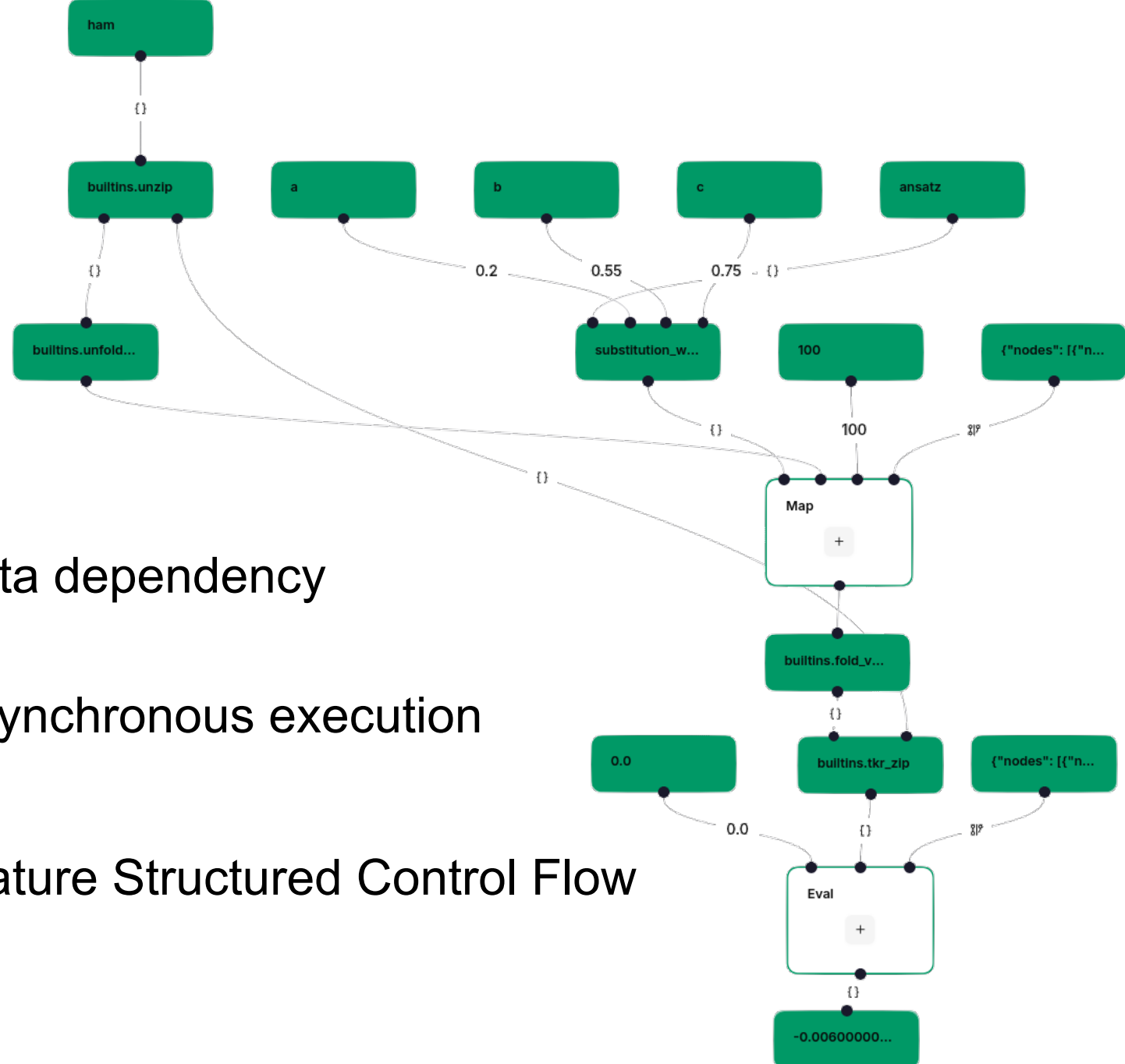
Nodes are computations

Edges carry data

Partial order of execution from data dependency

Allows parallel scheduling and asynchronous execution

Nodes can be Higher Order or feature Structured Control Flow



Runtime Visualization

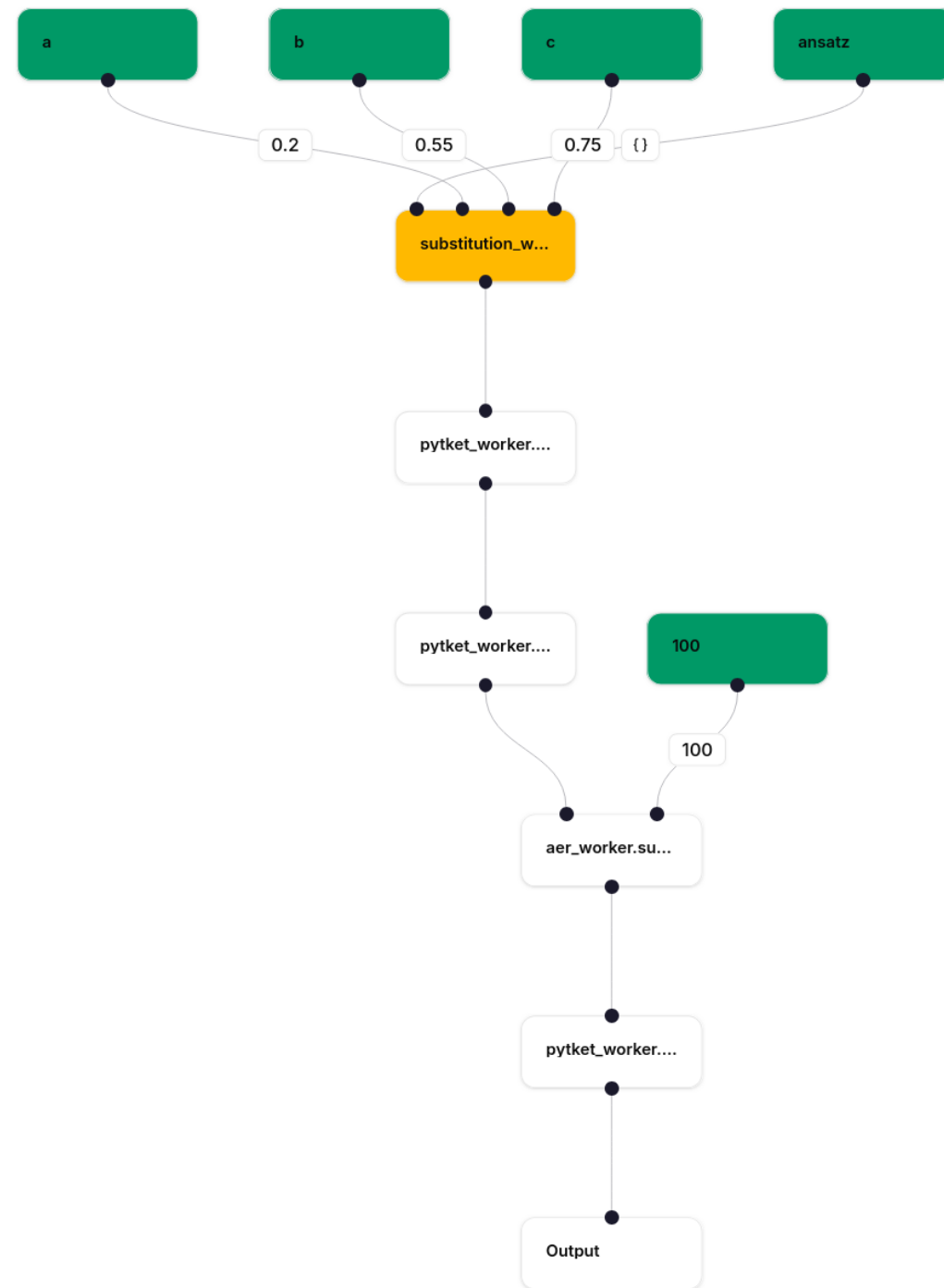
Visualizer will update live

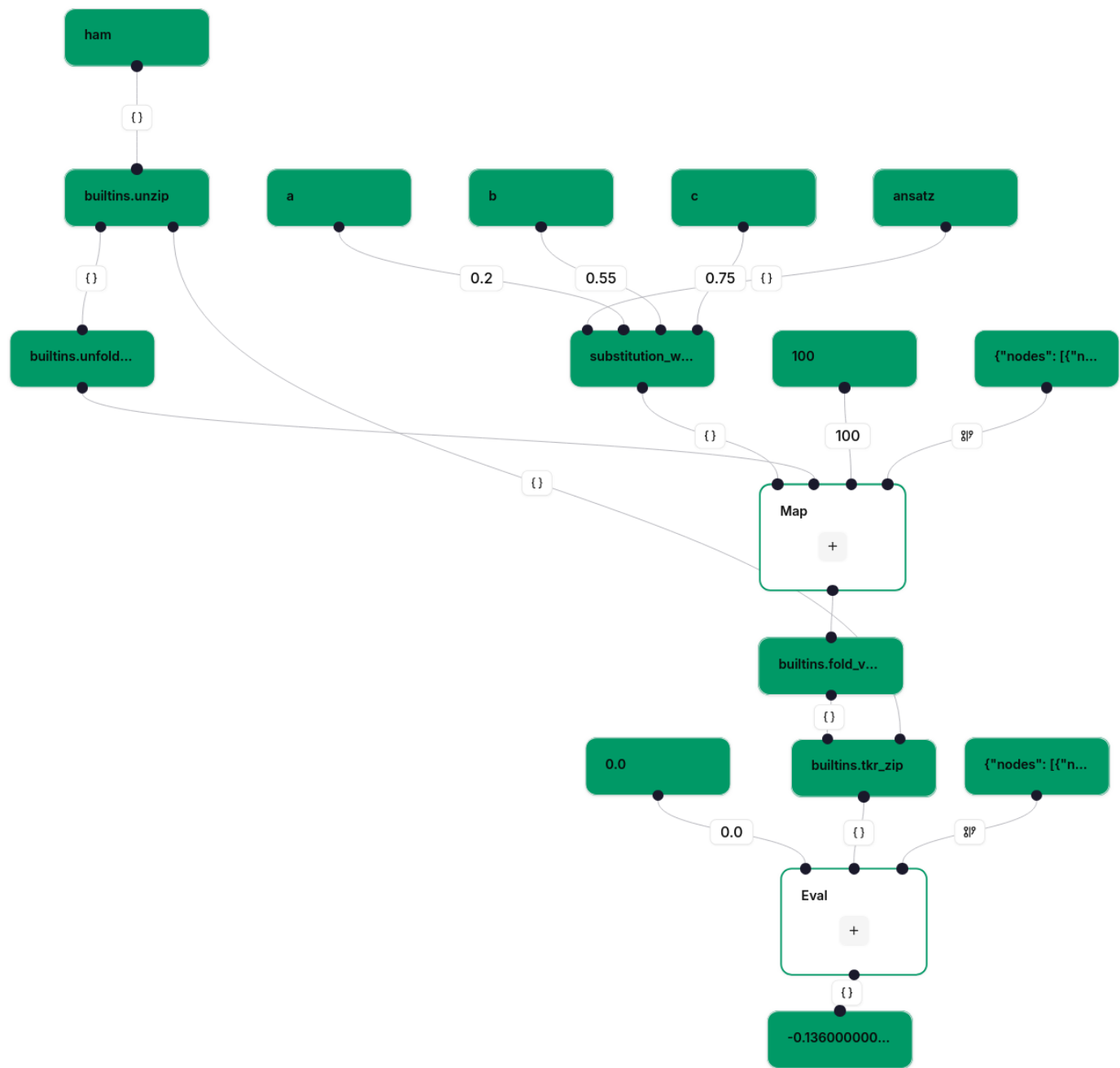
White means waiting

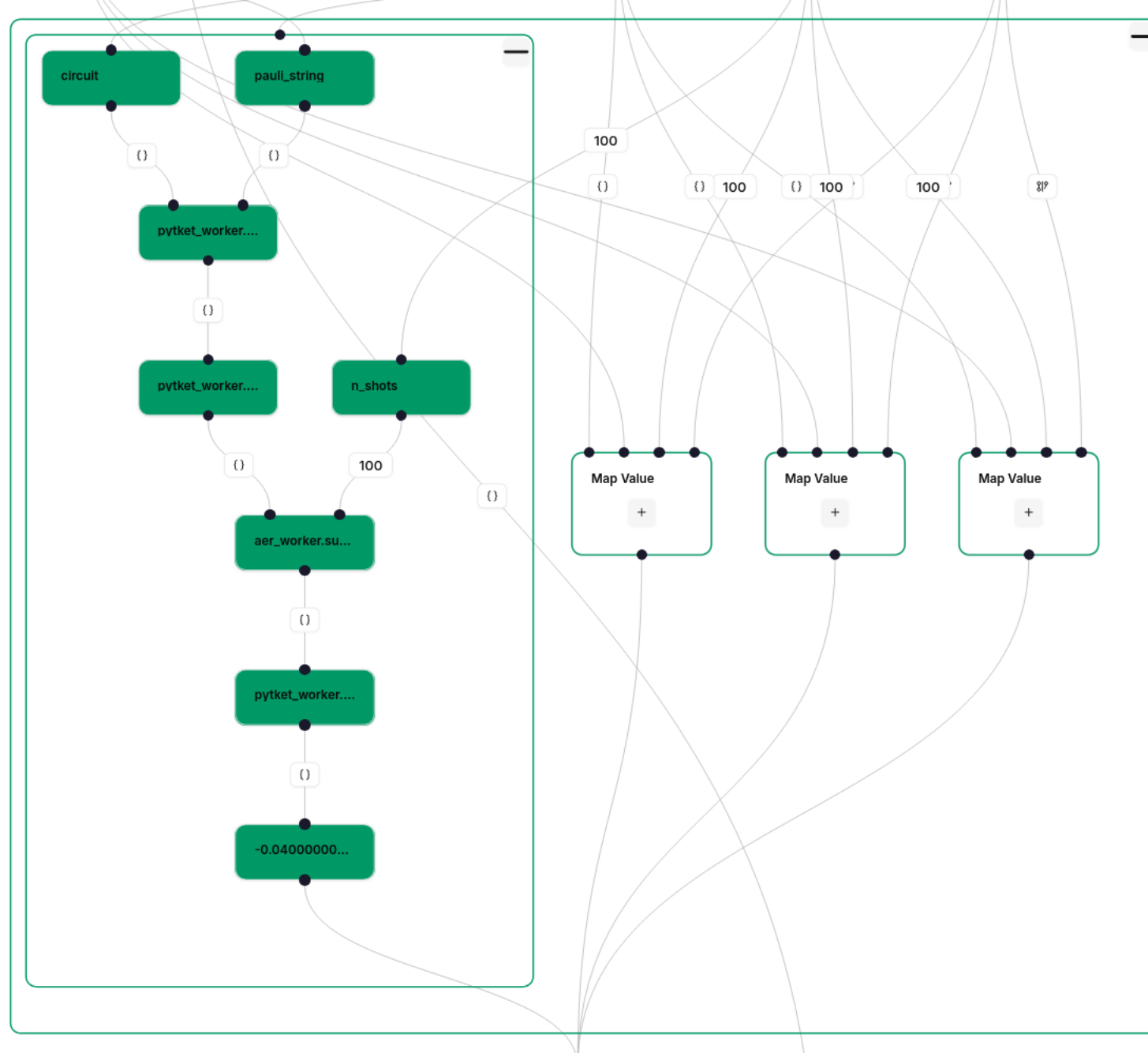
Green means done

Yellow means working

Red means error

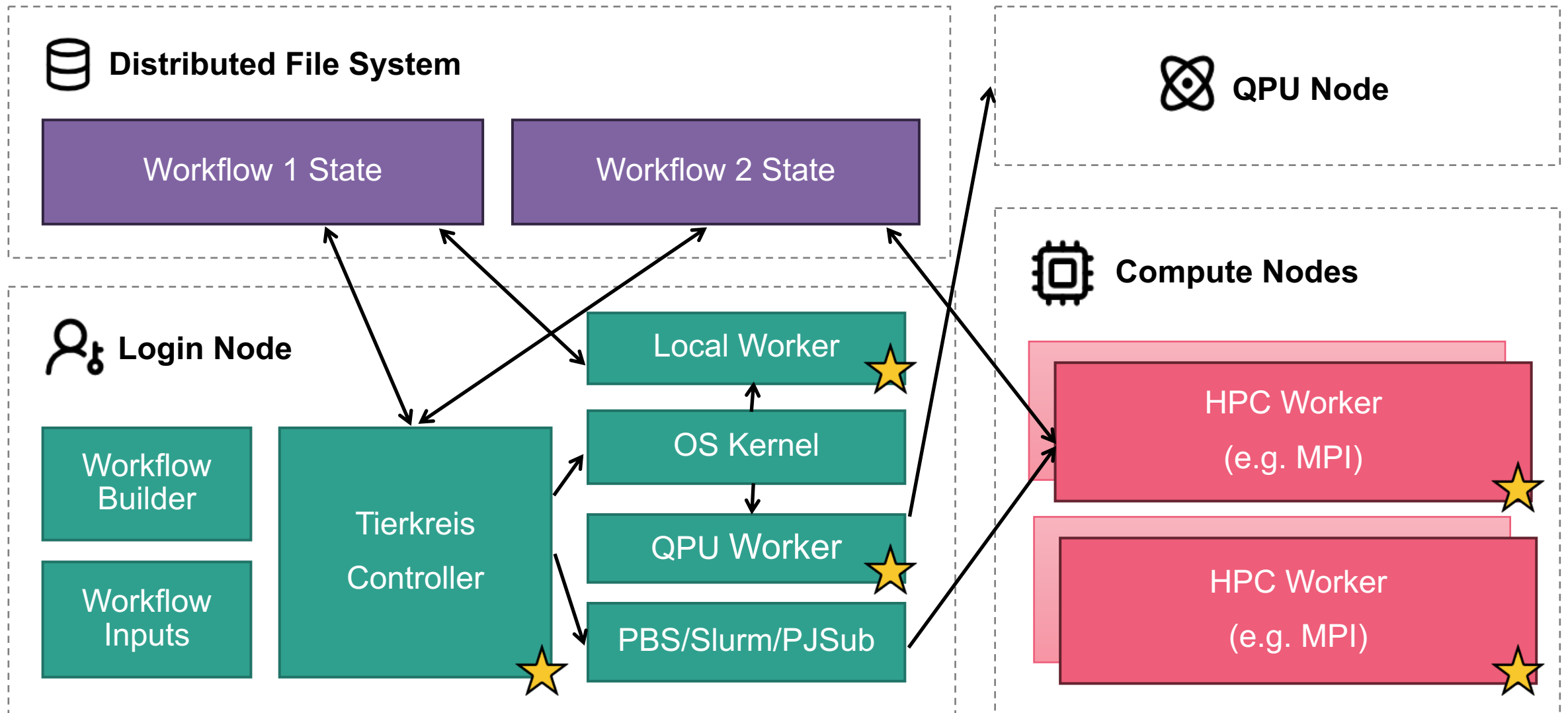






Tierkreis Execution Model

★ Tierkreis Processes



Workers

Tasks are performed by workers. Examples include:

- Built-in functions
- Pre-packaged libraries
- Wrapped HPC applications

Workers can publish a type signature to prevent runtime errors

Language agnostic, but we provide a python SDK

Checkpointing

Intermediate values between computations are stored in a checkpoint directory on the distributed file system

Intermediate values and logs can be inspected for debugging

Workflows can be interrupted and restored from the checkpoints

HPC Tasks

Tierkreis has an Executor interface that allows us to re-use existing HPC abstractions

Different Workload Managers can be supported by the system by implementation of Executors

The Fujitsu Job System, Slurm and PBS are supported in the main Tierkreis Package

Users can configure their resource requirements for a task (Number of Nodes, CPUs, Memory etc.)

ResourceSpec
nodes: 16
memory_gb: 128

MPISpec
max_proc_per_node: 1

UserSpec
mail: me@home.com

SLURM

```
#SBATCH --time=00:15:00
#SBATCH --nodes=16
#SBATCH --ntasks-per-node=1
```

...

Fujitsu Job System

```
#PJM -L "elapse=00:15:00"
#PJM -L "-node=16"
#PJM --mpi "max-proc-per-node=1"
```

...

PBS

...

Thank you